

GO ABSOLUTE MASTERY SYLLABUS

AN UNCOMPROMISED FIRST-PRINCIPLES STUDY INVENTORY

PHASE 1: PRIMITIVE SYNTAX & BASIC TYPES

- Lexical elements (identifiers, keywords, operators, delimiters, literals)
- Variable declarations (explicit, implicit short-assignment, zero-values)
- Constant declarations (untyped vs. typed constants, iota enumerations)
- Numeric types (int8-int64, uint8-uint64, uintptr, float32, float64, complex64, complex128)
- Boolean and primitive string types
- Control flow blocks (if, else, switch, initialization statements)
- For loops (classic three-component iteration, infinite execution loops)

PHASE 2: COMPOSED TYPES & MEMORY TOPOLOGY BASE

- Address pointers (dereferencing *T, address extraction &T, pointer restrictions)
- Stack allocation frame lifecycles vs. System Heap escape paths
- Fixed arrays (compile-time allocation limits, value-copy assignment behavior)
- Slices (Slice Header anatomy: backing array pointer, length descriptor, capacity field)
- Slice mutation loops (append execution paths, dynamic allocation doubling)
- Strings under the hood (immutable byte allocations, range loops over raw bytes vs. runes)
- Maps (hash table engine buckets, overflow keys, key-type constraints, value lookup safety)

PHASE 3: SUBROUTINES, CONTROL FLOW EXTENSIONS & METHODS

- Functions (multi-value return arguments, named return parameters)
- Variadic inputs, deferred execution blocks (defer stack, parameter snapshotting)
- Panic initialization vectors and emergency execution recovery loops (recover, recover boundaries)
- Struct layouts (field spacing configurations, anonymous fields, structural alignment memory rules)
- Methods (value receivers vs. pointer receivers, implicit pointer conversions)

PHASE 4: INTERFACES & TYPE POLYMORPHISM

- Interface values under the hood (iface vs eface, runtime data type pointers)
- Implicit structural implementation mechanics (decoupled duck-typing checks)
- Type assertions, type switches, empty interface wrappers (interface{})
- Interface compliance verification patterns at compile time

PHASE 5: CONCURRENCY PRIMITIVES & RUN-TIME SCHEDULING

- The Go M:N Scheduler (Goroutines, Logical Processors, Kernel Threads, Work-stealing)
- Mutual exclusion locks (sync.Mutex, sync.RWMutex, lock granularities)
- Atomic operations (sync/atomic package primitives, lock-free memory modification)
- Channels (unbuffered synchronization points vs. buffered queue networks)
- Channel pipeline controls (range traversal, structural channel closure rules)
- Multiplexing select frameworks (non-blocking cases, default fallbacks)
- Synchronization barriers (sync.WaitGroup, sync.Once, sync.Cond)
- Context propagation scopes (context.Context, timeouts, deadlines, cancel signals)

PHASE 6: GENERICS & METAPROGRAMMING

- Type parameters, type constraints, type sets, pre-declared constraints (any, comparable)
- Generic function signatures and structural generic type layouts
- Type approximation markers (~T operator constraints)
- Run-time reflection layers (reflect.Type, reflect.Value, structural tag parsing)
- Unsafe pointer manipulation blocks (unsafe.Pointer, uintptr transformations)

PHASE 7: OPTIMIZATION, INSTRUMENTING & PRODUCTION PATTERNS

- Tri-color concurrent mark-and-sweep garbage collection algorithms
- Memory reuse pools (sync.Pool allocations, garbage collection bypass optimization)
- Native benchmarking frameworks (testing.B, memory allocation counts, CPU tracking)
- Performance profiling modules (pprof CPU maps, heap allocation tracking, execution traces)
- Repository compilation flags (go build -gcflags="-m" for escape analysis audits)
- Graceful system termination loops (OS signal interception, asynchronous task draining)
- Domain invariant pre-flight validations and structured append-only serialization profiles